

1. AWS RDS

Amazon Relational Database Service (RDS) is a **managed cloud database service** provided by AWS. It simplifies the process of **setting up, operating, and scaling relational databases in the cloud**.

AWS automatically handles:

- Hardware provisioning
- Database setup
- Patching
- Backup
- Recovery
- Scaling
- Monitoring

Supported Database Engines

Amazon RDS supports multiple database engines:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- Microsoft SQL Server
- Amazon Aurora

Advantages of RDS

- Easy database setup
- Automatic backups
- High availability
- Security and encryption
- Automatic software patching
- Monitoring using CloudWatch
- Scalable storage and compute

2. AWS RDS Availability Zones and Local Zones

Availability Zones (AZ)

An **Availability Zone (AZ)** is a **physically separate data center within an AWS Region**.

Each AWS region contains **multiple availability zones** to provide **fault tolerance and high availability**.

Example:

- Region: **Asia Pacific (Mumbai)**
- Availability Zones:
 - ap-south-1a
 - ap-south-1b
 - ap-south-1c

Characteristics of Availability Zones

- Isolated infrastructure
- Independent power and networking
- High-speed connection between zones
- Designed for high availability

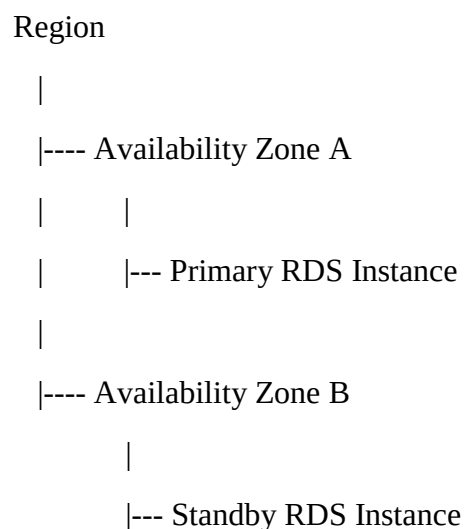
Role of AZ in RDS

When using RDS:

- The database runs in **one availability zone**
- A standby database can run in **another availability zone**

This ensures **failover during system failure**.

Example Architecture



Benefits

- Fault tolerance
- Disaster recovery
- High availability

Local Zones

Local Zones are AWS infrastructure extensions that place **compute, storage, and database services closer to end users**.

They are used when applications require:

- Ultra-low latency
- Local data processing
- Real-time applications

Example Use Cases

- Gaming
- Video streaming
- Real-time analytics
- Machine learning inference

Characteristics

- Located near large metropolitan areas
- Connected to parent AWS region
- Reduces network latency

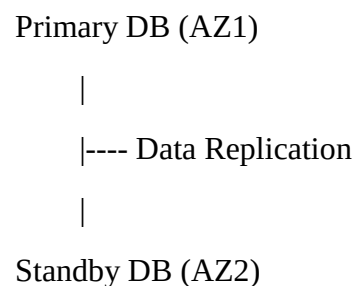
3. Multi-AZ Deployments in Amazon RDS

Multi-AZ deployment provides **high availability and automatic failover** for databases.

In this architecture:

- A **primary database** runs in one AZ
- A **standby replica** runs in another AZ

The standby database continuously synchronizes with the primary database.



Failover Process

If the primary database fails:

1. AWS automatically detects the failure
2. The standby database becomes the new primary database
3. Application reconnects automatically

Advantages

- High availability
- Automatic failover
- Data durability
- No manual intervention

DB Instance Billing for Amazon RDS

Amazon RDS pricing is based on several components.

1. DB Instance Cost

You pay for the **compute capacity** of the database instance.

Pricing depends on:

- Instance type
- Region
- Database engine
- Usage hours

Example instance types:

- db.t3.micro
- db.t3.small
- db.m5.large

2. Storage Cost

RDS charges for the amount of **storage allocated**.

Types of storage:

- General Purpose SSD
- Provisioned IOPS SSD
- Magnetic Storage

Example:

- 20 GB storage → monthly charge

3. Backup Storage

AWS provides **automatic backup storage**.

Charges depend on:

- Backup size
- Retention period

Backup retention can be:

- 1 to 35 days

4. Data Transfer Cost

Charges apply for:

- Data transfer outside AWS
- Cross-region data replication

5. Multi-AZ Cost

If Multi-AZ deployment is enabled:

- AWS creates **two database instances**
- Billing increases because of standby instance

6. Reserved Instance Pricing

AWS offers **Reserved DB Instances** for long-term usage.

Benefits:

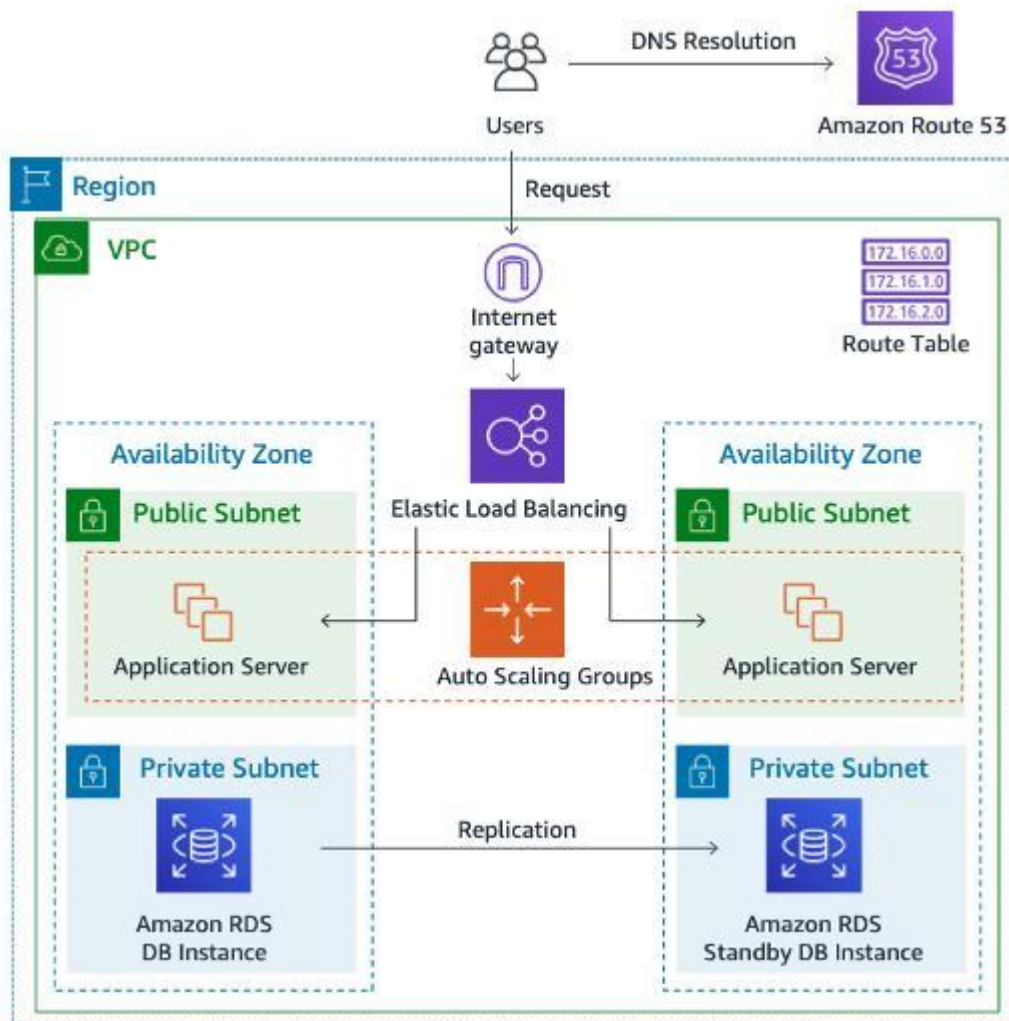
- Lower cost
- Up to **40–60% discount**

Types:

- 1-year reservation
- 3-year reservation

steps to set up and configure an Amazon RDS DB instance for a web application.

Setting up an Amazon RDS (Relational Database Service) instance for a web application involves provisioning the database engine, configuring networking for security, and establishing a connection from your application host.



1. Initiate Database Creation

Log in to the AWS Management Console and navigate to the **RDS Dashboard**.

Select Create Database: Choose **Standard create** for full control over configuration or **Easy create** for recommended best practices.

Choose Engine: Select a database engine such as **MySQL**, **PostgreSQL**, or **MariaDB**.

Select Template: Choose **Production** for high availability, **Dev/Test** for development, or **Free Tier** to use eligible resources without cost

2. Configure Settings and Instance Class

DB Instance Identifier: Provide a unique name for your instance (e.g., web-app-db).

Credentials: Set a **Master username** and a strong **Master password**. These will be used for your application's initial connection.

Instance Type: Select an instance class that fits your workload, such as db.t3.micro for small apps or testing.

Storage: Allocate storage (minimum 20 GiB for most engines) and consider enabling **Storage Autoscaling** to handle unpredictable data growth

3. Configure Connectivity and Security

This step is critical for ensuring your web application can reach the database securely.

Network Selection: Choose the Virtual Private Cloud (VPC) and subnets where your database will reside.

Public Access: Select **No** for production environments to keep the database private. Select **Yes** only if you need to connect directly from a local machine for testing.

VPC Security Groups: Create or select a security group that allows inbound traffic on the database port (e.g., 3306 for MySQL, 5432 for PostgreSQL) from your web application's security group or IP address.

Availability: For production, enable **Multi-AZ deployment** to provision a standby replica in a different Availability Zone for failover support.

4. Finalize Additional Configurations

Expand the **Additional configuration** section to set initial database properties.

Initial Database Name: Provide a name to automatically create a database upon launch.

Backup & Monitoring: Enable **Automated Backups** and **Enhanced Monitoring** to track database health and performance via Amazon CloudWatch.

Maintenance: Enable **Auto minor version upgrades** to keep your engine updated automatically

5. Connect the Web Application

Once the instance status is **Available**, locate the **Endpoint** in the "Connectivity & security" tab.

Update Application Config: In your web application's configuration file (e.g., `.env` or `config.php`),

enter the RDS **Endpoint**, **Port**, **Username**, and **Password**.

Verify Connection: Test the connection using a database client like MySQL Workbench or via your application's startup logs